EECS 3311 Software Design – Fall 2026

StudyBuddy

Stage 2 Implementation Report

Andrew Manga
November 9, 2026

1. Project Overview and Implementation Status

StudyBuddy is a desktop AI agent that helps students learn from their own course materials. Students upload their slides, notes and readings. The agent extracts their key concepts, answers questions with citations, generates flashcards and quizzes, grades answers, and tracks which concepts the student has mastered. A conversational Study Coach carries out multi-step study tasks on its own by planning and calling these capabilities as tools.

This report describes the implementation submitted in the public repository github.com/manseband/studybuddy. The design it implements is in report/Stage1.docx.

Target Users

The primary demographic for StudyBuddy is **university/college students** preparing for examinations in courses with substantial reading or lecture material. The system is a single-user desktop application with no accounts or login. A CLI alternative is available for more technically minded users.

AI/LLM Integration

- The model provider is Google Gemini, accessed through the Gemini API and integrated with LangChain4j's Gemini module. gemini-3.1-flash-lite handles generation, grading and the Study Coach's planning and tool calling. gemini-embedding-001 produces the embeddings used for retrieval. Model IDs are set in configuration, not in code.
- All models sit behind StudyBuddy's own LLMClient interface, and only the studybuddy.ai package imports LangChain4j. In testing, a FakeLLMClient returns scripted responses, so no test reaches the network. With no API key, a NoKeyLLMClient is used: deterministic features keep working, search falls back to keyword ranking, and AI features tell the user how to add a key.
- The API key is read from the GEMINI_API_KEY environment variable or a local .env file, which is gitignored. Only .env.example, containing placeholders, is committed.

Agent Behaviour

The AI component does more than forward a prompt to a model:

- **Retrieval.** Questions, quizzes, flashcards and explanations are grounded in passages retrieved from the course's own materials and cited by material and page.
- **Planning and tool use.** The Study Coach runs a reason-act loop. The model chooses among five tools (search materials, get weak concepts, generate a quiz, generate flashcards, save a preference), and each result feeds the next step.
- **Memory.** The conversation is remembered for the last 20 turns, and stated preferences are stored in SQLite so they outlast the session.
- **Decision making.** The Coach asks which course is meant when that is unclear, matches quiz difficulty to the student's mastery unless told otherwise, and declines to write answers to graded work.

- **Bounded autonomy.** A run stops after eight tool calls or three invalid calls in a row. Tools may read and create data, but never delete or modify existing data, and every step is shown to the student as it runs.

Architecture

The system uses the layered architecture from Stage 1, comprised of five main layers.

Presentation Layer	A Swing GUI and a picocli CLI with no business logic. Both call the same StudyAgentFacade, so every feature behaves identically from either interface. Every CLI command supports --json.
Application Layer	StudyAgentFacade is the lone entry point. It delegates to one service per feature area (MaterialService, LearningContentService, FlashcardService, QuizService, MasteryService, ProgressService), which hold the business rules. ApplicationFactory wires the concrete classes together.
Agent Layer	AgentController runs the Study Coach. Its Planner runs a bounded reason-act loop with the LLM. ToolManager validates and executes tool calls, and ConversationMemory and AgentTrace record turns and steps. Each tool is a thin wrapper around an existing service, so the agent reuses the same tested code as the GUI.
AI Integration Layer	The LLMClient interface hides which model provider is in use. PromptBuilder assembles prompts, ResponseParser validates structured LLM output, and RetrievalStrategy finds relevant passages.
Domain & Persistence Layer	Plain domain objects (Course, Material, Quiz, Question, Flashcard, ConceptMastery, etc.) and repositories that store them in a local SQLite database.

Current Working Functionality

All ten features described in the Stage 1 report run end to end from both interfaces. The GUI has one tab per feature area, and the CLI is started with the studybuddy command.

Feature	GUI tab	CLI command	Type
F01 Course and Material Management	Materials	course, material	Deterministic
F02 Concept Extraction	Concepts	concepts	AI-based
F03 Question Answering over Course Materials	Ask	ask	Hybrid
F04 Flashcard Generation and Review	Flashcards	flashcards	Hybrid
F05 Quiz Generation	Quizzes	quiz generate, list, show	AI-based
F06 Quiz Grading	Quizzes	quiz take, grade, results, regrade, dispute	Hybrid
F07 Answer Explanation	Quizzes	explain	AI-based
F08 Weak-Concept Identification	Progress	weak-concepts	Hybrid
F09 Study Coach	Coach	coach	AI agent
F10 Progress Tracking and Study History	Progress	progress, history	Deterministic

Limitations and Incomplete Functionality

No feature from the Stage 1 report is incomplete. F07, F08, F09 and F10 were adjusted during implementation, and Section 4 records each change (C26, C23 and C24, C25 and C29, and C27 respectively). The following limitations remain in the submitted application:

- Materials must be text-based PDF, TXT or Markdown files in English. There is no OCR, so a scanned PDF is stored with the status "No text found" and cannot be searched.
- AI features need an internet connection and a Gemini API key.
- Quiz answer keys are written by the model. There is no calculation question type, and a numeric answer key is not checked independently.
- A preference saved through the Study Coach cannot be removed from within the application.
- Study Coach conversations are kept in memory for the session. Only the first line of each conversation is stored, for the study history.

2. Feature to Implementation Mapping

Each feature below is given as a mapping table, followed by how its classes collaborate from user input to final result. File paths are relative to src/main/java/studybuddy/. Change IDs (C01 to C31) refer to Section 4. All ten features come from Stage 1 and there are no new or replaced features. Every feature is reached through StudyAgentFacade, so a GUI panel and a CLI command for the same feature call the same facade method.

F01 — Course and Material Management

Stage 1 feature/reference	F01
Status or change	Implemented. Details in C10, C11, C12, C18, C19.
Implemented classes/files and responsibilities	**presentation/gui/MaterialsPanel, MainWindow; presentation/cli/CourseCommand, MaterialCommand:** take the course name or file and show the result. **application/StudyAgentFacade:** the entry point for both interfaces. **application/MaterialService:** validates files, detects duplicates, splits text into chunks, saves and indexes the material, and publishes events. **ai/DocumentParser:** extracts and cleans page text from PDF, TXT and Markdown files. **ai/RetrievalStrategy, EmbeddingRetriever, KeywordRetriever:** index chunks for later search. **persistence/CourseRepository, MaterialRepository, Database:** store courses, materials and chunks; one transaction per upload. **application/EventBus:** tells MainWindow to refresh the panels.
Key methods	createCourse(), uploadMaterial(), validateFile(), extractPages(), splitIntoChunks(), index(), load(), refreshEmbeddings(), renameMaterial(), removeMaterial(), deleteCourse(), publish(), onStudyEvent()

Design patterns	Facade, Strategy, Observer
Video link	https://drive.google.com/file/d/1RgyOD2rMaxIQczI9JrvJszfDBlFBEd16/view?usp=drive_link

When the student uploads a file, the facade passes it to MaterialService, which checks its type and size and compares its SHA-256 hash with the course's files, so a duplicate can be replaced or skipped. DocumentParser extracts the text page by page and removes repeated headers and footers. The service groups pages into chunks that record their page range, saves the material and chunks in one transaction, and indexes them through the RetrievalStrategy. Finally, it publishes an event on the EventBus so MainWindow refreshes the materials list. A file with no readable text is saved with the status "No text found" and is not indexed. If the embedding service fails, the material is still saved and can be searched by keyword. **Classification: Deterministic.** Embeddings are only used to index passages, and keyword search works without them.

F02 — Concept Extraction

Stage 1 feature/reference	F02
Status or change	Implemented. Details in C17, C28, C30.
Implemented classes/files and responsibilities	**presentation/gui/ConceptsPanel; presentation/cli/ConceptsCommand:** start extraction, show candidates for review, and manage saved concepts. **application/LearningContentService:** builds the requests, removes duplicates and saves the result. **ai/PromptBuilder, LLMClient, GeminiClient, ResponseParser:** build the prompt, call the model and validate the JSON reply. **persistence/ConceptRepository:** stores concepts, prerequisites and material links; merges concepts in one transaction. **domain/Concept:** normalizes names for duplicate detection.
Key methods	extractConcepts(), requestConcepts(), groupByWordBudget(), build(), chat(), requestJson(), saveConcepts(), updateConcept(), mergeConcepts(), deleteConcept()
Design patterns	Facade, Builder, Strategy, Adapter
Video link	https://drive.google.com/file/d/19VvBUBV-e47xMJVX4pg-9yPnl3pS_EFg/view?usp=drive_link

LearningContentService groups the material's chunks to fit a prompt word budget, builds an extraction prompt for each group and sends it through LLMClient. ResponseParser turns each reply into candidate concepts, with one repair request if the JSON is malformed. The service merges duplicates with the course's existing concepts and returns the candidates without saving them. The student edits or removes candidates and confirms, and only then does saveConcepts() store them. These concepts become the tags used by quizzes, flashcards and mastery tracking. Deleting a concept leaves its questions and cards untagged rather than removing them. **Classification: AI-based.**

F03 — Question Answering over Course Materials

Stage 1 feature/reference	F03
Status or change	Implemented. Details in C10, C15, C16.
Implemented classes/files and responsibilities	**presentation/gui/AskPanel; presentation/cli/AskCommand:** take the question and show the answer and its sources. **application/LearningContentService:** retrieves passages, requests the answer and saves it to the history. **ai/RetrievalStrategy:** finds the most relevant passages above a relevance threshold. **ai/PromptBuilder, GeminiClient, ResponseParser:** build the prompt from numbered excerpts, call the model and check the citations. **persistence/QaHistoryRepository; domain/Answer, QaEntry, Citation:** store and represent answers.
Key methods	askQuestion(), answerQuestion(), retrieve(), withExcerpts(), build(), chat(), requestJson(), validateCitations(), getQuestionHistory()
Design patterns	Facade, Strategy, Builder, Adapter
Video link	https://drive.google.com/file/d/17FxRR8xniGjY8xbYvJgPmJty9LO041en/view?usp=drive_link

LearningContentService first asks the RetrievalStrategy for the passages most relevant to the question. If none are relevant enough, it returns a "not covered" answer, or a clearly labelled general-knowledge answer when the student allows one. Otherwise, PromptBuilder packages the numbered passages with the question, and GeminiClient converts the request for LangChain4j's ChatModel. ResponseParser parses the answer and removes any citation that does not match a passage it was given. The answer is saved to the history. If the model fails, the retrieved passages are still shown. In the GUI, double-clicking a source shows its excerpt. **Classification: Hybrid.** Retrieval and citation checking are deterministic, and the answer text is written by the LLM.

F04 — Flashcard Generation and Review

Stage 1 feature/reference	F04
Status or change	Implemented, with deck deletion added. Details in C24, C29.
Implemented classes/files and responsibilities	**presentation/gui/FlashcardPanel; presentation/cli/FlashcardsCommand:** choose the scope, preview and edit a deck, run reviews. **application/FlashcardService:** generates, validates and saves decks, finds due cards and records ratings. **application/StudyScope:** gathers the excerpts for the chosen materials or concepts (shared with QuizService). **domain/Flashcard:** holds the Leitner box and next due date. **persistence/FlashcardRepository; domain/FlashcardReview:** store cards and ratings.

Key methods	generateFlashcards(), generateCards(), saveDeck(), getDueCards(), getDueCardsInDeck(), recordReview(), review(), deleteDeck(), nextDueDate()
Design patterns	Facade, Strategy, Builder, Observer
Video link	https://drive.google.com/file/d/1s528BwL6EAiA_69QviwCCLh6b1lhpru0/view?usp=drive_link

For generation, FlashcardService uses StudyScope to gather excerpts for the chosen materials or concepts and makes one model call. Cards that are empty, too long, tied to an unknown excerpt or repeated are dropped, and the student is told when fewer cards than requested remain. The unsaved deck is returned so the student can edit it, and saveDeck() stores the cards together under a new deck id. For review, the service loads the cards that are due. Each rating goes to Flashcard.review(), which moves the card between spaced-repetition boxes and sets its next due date. The card and a review record are saved together, and the rating then counts as mastery evidence for the card's concept. Deleting a deck keeps its ratings. **Classification: Hybrid.** Cards are written by the LLM, and scheduling is deterministic.

F05 — Quiz Generation

Stage 1 feature/reference	F05
Status or change	Implemented. Details in C05, C22, C24.
Implemented classes/files and responsibilities	**presentation/gui/QuizPanel; presentation/cli/QuizCommand:** take the scope, count, question types and difficulty, and open the finished quiz. **application/QuizService:** requests, validates and regenerates questions, and saves the quiz. **application/StudyScope:** gathers the excerpts for the chosen scope. **application/QuestionFactory; domain/Question and subclasses:** build and validate each question. **persistence/QuizRepository:** stores quizzes and answer keys, and supplies recent question stems so repeats can be rejected.
Key methods	generateQuiz(), splitAcrossTypes(), requestQuestions(), toQuestion(), createQuestion(), isValid(), findRecentStems(), save()
Design patterns	Facade, Factory, Strategy, Builder
Video link	https://drive.google.com/file/d/1z44zFqFwg1hX-y89M0f-X3_p_dNJaY3D/view?usp=drive_link

QuizService uses StudyScope to gather excerpts for the quiz's scope, caps the question count by the amount of text available and splits the count across the chosen types. The LLM returns the questions as structured data. For each one, the service checks the excerpt number, fixes the page to that excerpt and matches the concept name. QuestionFactory turns the reply into the right Question subclass, and isValid() checks it. Invalid questions and repeats of the last five quizzes are regenerated up to twice or dropped, and the finished Quiz is saved and opened in the panel. **Classification: AI-based.** The questions are written by the LLM and validated by code.

F06 — Quiz Grading

Stage 1 feature/reference	F06
Status or change	Implemented. Details in C05, C22.
Implemented classes/files and responsibilities	**presentation/gui/QuizPanel; presentation/cli/QuizCommand:** collect answers and show results, re-grading and disputes. **application/QuizService:** picks a grader per answer, saves the attempt and publishes the event. **grading/GradingStrategy, RuleBasedGrader, LLMRubricGrader:** grade by answer key or against the rubric. **domain/QuizAttempt, GradeResult:** hold the attempt's state and each result. **persistence/AttemptRepository:** stores attempts and grades.
Key methods	submitQuiz(), submitAttempt(), gradeAnswer(), grade(), submit(), recordResult(), regradePending(), overrideGrade(), publish()
Design patterns	Strategy, Observer, Facade
Video link	https://drive.google.com/file/d/1-88aNDoQvLIArPuq-KdLW7YLq938aB9_/view?usp=drive_link

QuizService.submitAttempt() picks a grading strategy for each answer. RuleBasedGrader compares multiple-choice and true/false answers with the answer key, while LLMRubricGrader asks the LLM to score short answers against the rubric and give feedback. Blank answers score zero with no model call. If AI grading fails, the answer is marked pending and regradePending() can retry it later. The attempt is saved, and a QUIZ_SUBMITTED event is published so progress views refresh. The student can dispute any AI-assigned grade through overrideGrade(). **Classification: Hybrid.**

F07 — Answer Explanation

Stage 1 feature/reference	F07
Status or change	Modified. No follow-up question (C26).
Implemented classes/files and responsibilities	**presentation/gui/QuizPanel; presentation/cli/ExplainCommand:** request the explanation and show it with the question. **application/StudyAgentFacade:** finds the question and the student's answer, and rejects answers still pending. **application/LearningContentService:** gathers the source excerpts, requests the explanation and caches it. **ai/RetrievalStrategy, PromptBuilder, LLMClient, ResponseParser:** retrieve supporting passages, build the prompt, call the model and validate the reply. **persistence/ExplanationRepository; domain/Explanation:** cache one explanation per attempt and question.
Key methods	explainAnswer(), explanationExcerpts(), requestExplanation(), retrieve(), build(), chat(), clearExplanation()

Design patterns	Facade, Builder, Strategy, Adapter
Video link	https://drive.google.com/file/d/1mXrjssd6hehjI0-fzBRq7pfI6yA_qmPJ/view?usp=drive_link

The facade finds the question and the student's answer and passes them to LearningContentService.explainAnswer(). A cached explanation is returned immediately. Otherwise the service uses the question's source passage plus up to two retrieved passages, builds an explanation prompt and calls the LLM. A misconception is kept only for a wrong, non-blank answer. The parsed explanation is cached with the attempt, so asking again does not call the model. If the source has since been deleted, the explanation relies on the answer key alone and says so. Disputing a grade clears that question's cached explanation. **Classification: AI-based.**

F08 — Weak-Concept Identification

Stage 1 feature/reference	F08 (Stage 1 name: Weak-Topic Identification)
Status or change	Modified. Renamed to weak concepts (C23); no practice shortcuts on the Progress tab (C24).
Implemented classes/files and responsibilities	**presentation/gui/ProgressPanel; presentation/cli/WeakConceptsCommand:** show the ranked concepts and request a diagnosis. **application/MasteryService:** calculates scores and trends, flags weak concepts and requests the diagnosis. **domain/ConceptMastery, Trend:** represent one concept's result. **persistence/AttemptRepository, QuizRepository, ConceptRepository, FlashcardRepository:** supply the evidence. **ai/LLMClient, ResponseParser:** used only for the optional diagnosis.
Key methods	getMastery(), getWeakConcepts(), computeMastery(), masteryScore(), trend(), diagnose(), diagnoseWeakConcepts()
Design patterns	Facade, Strategy, Builder
Video link	https://drive.google.com/file/d/1TOZIn4fsf2VBEYOKSLTthuHszY3MihO2/view?usp=drive_link

MasteryService.computeMastery() loads the course's graded answers and flashcard ratings and computes a score for each concept, giving more weight to recent and harder questions. Concepts below the threshold of 60 are flagged as weak, and concepts with fewer than three pieces of evidence are marked as having insufficient data instead. Scores are calculated on demand and never stored. getWeakConcepts() returns the weak concepts ranked, and diagnose() asks the LLM to describe patterns in recent mistakes. If the diagnosis fails, the scores still show. **Classification: Hybrid.** Scoring is deterministic, and the diagnosis is written by the LLM.

F09 — Study Coach

Stage 1 feature/reference	F09
Status or change	Modified. Coach model changed, preference tool added (C25); links to created items removed (C29).
Implemented classes/files and responsibilities	**presentation/gui/CoachPanel; presentation/cli/CoachCommand:** take the goal and show each step as it runs. **agent/AgentController:** builds the context, runs one message and updates memory. **agent/Planner:** runs the bounded reason-act loop with the LLM. **agent/ToolManager, AgentTool:** check a call's arguments against the tool's schema, run it and record it. **agent/SearchMaterialsTool, GetWeakConceptsTool, GenerateQuizTool, GenerateFlashcardsTool, SavePreferenceTool:** wrap existing services and store preferences. **agent/ConversationMemory, AgentTrace; persistence/PreferenceRepository:** hold the conversation, the steps and saved preferences.
Key methods	askCoach(), handleMessage(), run(), buildPrompt(), chat(), getToolSpecs(), execute(), validateArgs(), addTurn(), startStep(), finishStep()
Design patterns	Command, Facade, Strategy, Builder, Adapter, Observer
Video link	https://drive.google.com/file/d/17RCoJIDfhiUHBQLTR-tr1ivJvGobppDo/view?usp=drive_link

AgentController builds the context (date, courses, the selected course's materials and concepts, saved preferences) and hands the goal to the Planner. At each step, the Planner calls the LLM with the conversation and the tool descriptions from ToolManager. If the reply requests a tool, ToolManager checks the arguments, runs the matching AgentTool (which calls an existing service such as MasteryService) and records the step. The result goes back to the LLM. The loop ends when the LLM gives a final answer, after eight tool calls, or after three invalid calls in a row. When it stops early, the student is told what was completed. If the model fails partway, work already done is kept and listed. **Classification: AI agent.**

F10 — Progress Tracking and Study History

Stage 1 feature/reference	F10
Status or change	Modified. No dashboard or score chart (C27).

Implemented classes/files and responsibilities	**presentation/gui/ProgressPanel, MainWindow; presentation/cli/ProgressCommand, HistoryCommand:** show the summary, mastery and filterable history, and export the report. **application/ProgressService:** builds the report and history from stored records, and listens for new coach conversations. **application/ProgressReportWriter:** writes the report as Markdown or CSV. **application/EventBus:** notifies MainWindow and ProgressService. **persistence/CoachConversationRepository and the feature repositories:** supply the dated records.
Key methods	publish(), onStudyEvent(), refresh(), getProgress(), getHistory(), currentStreak(), exportReport(), toMarkdown(), toCsv()
Design patterns	Observer, Facade
Video link	https://drive.google.com/file/d/1GhCmjxU_tNwq9BItWallPFxMtsu9vWar/view?usp=drive_link

Whenever a service records new activity, it publishes an event on the EventBus. MainWindow receives the event and tells the panels to refresh. The panel requests a progress report from the facade, and ProgressService builds it from the stored quiz attempts, flashcard reviews, questions and coach conversations, without storing anything it calculates. The Progress tab shows the average score and study streak above the concept mastery and the filterable study history. The report can also be exported to a Markdown or CSV file. **Classification: Deterministic.**

3. Implemented Design Patterns

All seven design patterns from Stage 1 are implemented, with the same participating classes and roles. No pattern was added, replaced or removed. For each pattern, there is a list of participating classes and their roles as seen in the UML class diagram, followed by the source files and methods involved, and any difference from Stage 1. File paths are relative to src/main/java/studybuddy/.

Facade

- StudyAgentFacade → the facade.
- MainWindow, the FeaturePanel subclasses and StudyAgentCli → clients.
- MaterialService, LearningContentService, FlashcardService, QuizService, MasteryService, ProgressService and AgentController → the subsystem.

Source files and methods: application/StudyAgentFacade.java, with one method per user action, such as uploadMaterial(), askQuestion(), generateQuiz(), submitQuiz(), askCoach() and getProgress().

Both the GUI and the CLI must offer every feature, but the logic is spread across six services and the agent. The facade gives one simple entry point per feature, so both interfaces call the same methods, behave identically and contain no business logic. Without it, each interface would depend on every service and repeat the same orchestration, so the GUI and CLI would gradually drift apart.

Compared with Stage 1: Matches Stage 1. The facade gained methods for the management actions each feature needed (C11, C13), and ApplicationFactory builds it (C13).

Strategy

- LLMClient → a strategy interface, implemented by GeminiClient, NoKeyLLMClient and FakeLLMClient.
- RetrievalStrategy → a strategy interface, implemented by EmbeddingRetriever and KeywordRetriever.
- GradingStrategy → a strategy interface, implemented by RuleBasedGrader and LLMRubricGrader.
- The services, the Planner and LLMRubricGrader → contexts of LLMClient. MaterialService and the content services → contexts of RetrievalStrategy. QuizService → the context of GradingStrategy.

Source files and methods: ai/LLMClient.java (chat()), ai/RetrievalStrategy.java (index(), load(), retrieve(), removeMaterial()), grading/GradingStrategy.java (grade()), and ApplicationFactory.create() and createGraders(), where the concrete strategies are chosen. QuizService.gradeAnswer() picks a grader by question type.

Several behaviours have interchangeable algorithms (which model generates text, how passages are retrieved, how an answer is graded). Callers depend only on the interface, so switching provider, retrieval method or question type means adding a class rather than editing conditionals in every service. It also makes unit tests deterministic, because the fake client and keyword retriever can be swapped in.

Compared with Stage 1: Matches Stage 1. NoKeyLLMClient is one extra LLMClient implementation for running without a key (C16). EmbeddingRetriever falls back to keyword search when embeddings are unavailable (C12).

Adapter

- LLMClient → the target interface.
- GeminiClient → the adapter.
- LangChain4j's ChatModel → the adaptee.
- The services and the Planner → clients.

Source files and methods: ai/GeminiClient.java: chat() calls toChatRequest() and toLLMResponse(), and toToolSpecification() converts our tool schemas.

StudyBuddy needs a small, provider-neutral interface, while LangChain4j has its own message and response types. GeminiClient converts between the two, so no library or provider classes leak outside the AI integration layer. Changing provider means writing a new adapter. Without one, every service would import library types, and a library upgrade or provider change would ripple through the whole system.

Compared with Stage 1: Matches Stage 1. The adapter also holds the retry policy (C09) and passes Gemini's thought signatures back unchanged (C25).

Command

- AgentTool → the command interface.
- SearchMaterialsTool, GetWeakConceptsTool, GenerateQuizTool, GenerateFlashcardsTool and SavePreferenceTool → concrete commands.
- The services they call → receivers.
- ToolManager → the invoker.
- Planner → the client, issuing the calls the LLM chooses.

Source files and methods: agent/AgentTool.java (getName(), getDescription(), getParameterSchema(), execute()), the five tool classes in agent/, and agent/ToolManager.java (register(), getToolSpecs(), execute(), validateArgs()).

The Study Coach picks actions at runtime from a tool name and arguments produced by the LLM. Wrapping each action as an object with a name, description, parameter schema and execute() method allows the ToolManager to validate, run and record every call in one place. New tools are added by registering a class. Without this, the agent would need an ever-growing switch statement mapping names to service calls, with validation and logging repeated in each branch.

Compared with Stage 1: Matches Stage 1. GetWeakTopicsTool is named GetWeakConceptsTool (C23), and SavePreferenceTool is an additional command (C25).

Observer

- EventBus → the subject.
- StudyEventListener → the observer interface.
- MainWindow → a concrete observer, which refreshes its panels.
- ProgressService → a concrete observer, which records new coach conversations.
- The services and AgentController → publish events through the EventBus.

Source files and methods: application/EventBus.java (subscribe(), unsubscribe(), publish()), application/StudyEventListener.java (onStudyEvent()), MainWindow.onStudyEvent() and ProgressService.onStudyEvent().

When a quiz is graded or a material is uploaded, several views need to update. Services publish an event without knowing which views exist, so the application layer stays independent of Swing, and new views can subscribe without changing any service. Without it, services would need references to GUI panels (the wrong dependency direction), or the GUI would have to keep polling for changes.

Compared with Stage 1: Matches Stage 1. ProgressService is a second listener (C27).

Builder

- PromptBuilder → the builder.
- LLMRequest → the product.
- LearningContentService, QuizService, FlashcardService, MasteryService, LLMRubricGrader and Planner → clients.

Source files and methods: ai/PromptBuilder.java: system(), withExcerpts(), withInstruction(), withOutputSchema(), withTools(), withHistory(), withUserMessage() and build().

Each prompt combines optional parts (system and task instructions, an output format, tool descriptions, etc.). Different features need different combinations. PromptBuilder assembles them step by step with chained calls ending in build(), so prompt formatting resides in one place and every request is consistent. Without it, each feature would build prompts by its own string concatenation, which makes them inconsistent and hard to adjust when testing reveals a problem.

Compared with Stage 1: Matches Stage 1. withHistory() and withUserMessage() were added for the agent loop (C08).

Factory

- QuestionFactory → the creator, through createQuestion(type, data).
- Question → the abstract product.
- MultipleChoiceQuestion, TrueFalseQuestion and ShortAnswerQuestion → concrete products.
- QuizService → the client.

Source files and methods: application/QuestionFactory.java (createQuestion()), domain/Question.java and its three subclasses, and QuizService.toQuestion().

The LLM returns quiz questions as structured data of different types. The factory turns each one into the right Question subclass, so QuizService works only with the abstract Question. Adding a question type means adding a subclass and one case in the factory. Without it, the code that builds each question type from model output would be scattered through quiz generation.

Compared with Stage 1: Matches Stage 1. Saved quizzes are rebuilt from their own columns by QuizRepository, so the factory is used for model output only.

UML Class Diagram

See /uml/ClassDiagram.svg. It was updated to reflect the implemented system.

UML Use-Case Diagram

See /uml/UseCaseDiagram.svg.

UML Sequence Diagrams

See /uml/SD{01-06}_{SequenceName}.png. They were updated to match the implemented message flows.

4. Changes and Rationale

This section is the consolidated record of differences between the Stage 1 design and the implementation, using the structure Original design → Implemented change → Reason → Impact. No feature was added or replaced, and no design pattern was added, replaced or removed, so most entries are additions the class diagram did not show.

Feature changes	C23 and C24 (F08), C25 and C29 (F09), C26 (F07), C27 (F10), and behaviour details in C17, C18, C22, C24, C28, C29, C30
Architecture changes	C03, C12, C13, C19, C20
Class and method changes	C01, C04 to C11, C14, C15, C16, C21, C22, C24, C25
Pattern changes	None. C12, C16 and C27 extend how existing patterns are used.
Build and tooling	C02, C31

C01 — Configuration loader

Original design	The design said model IDs and the API key come from configuration, but showed no class for it.
Implemented change	Added one small configuration component that combines default settings, a local file and environment variables.
Reason	Several layers need settings, and the API key must stay out of the source code.
Impact	Supporting class only. No feature or design pattern changes.

C02 — Maven Wrapper

Original design	A Maven build, with no wrapper mentioned.
Implemented change	Added the Maven Wrapper.
Reason	The project can be built without installing Maven.
Impact	Build tooling only.

C03 — Course composition stored as references

Original design	A course directly contains its materials, concepts, quizzes and flashcards.
Implemented change	Each item refers to its course and is loaded when needed. Deleting a course still deletes everything in it.
Reason	Loading everything whenever a course is read would be slow and would tie the repositories together.
Impact	Domain and persistence only. No feature or pattern change.

C04 — Extra domain attributes not in the class diagram

Original design	The class diagram listed the basic attributes of each domain class.
Implemented change	Added attributes the features needed, such as page counts, concept tags and deck membership, and small supporting value types.
Reason	Features display or depend on this information.
Impact	Domain only. No class renamed and no pattern change.

C05 — Quiz domain additions

Original design	Questions, attempts and grade results held only basic data.
Implemented change	Questions record their concept and source, attempts track the result of each question, and grades can be pending or overridden.

Reason	Mastery tracking, explanations, re-grading and disputes depend on this information.
Impact	Supports F05, F06, F07 and F08. No pattern change.

C06 — Database setup and supporting tables

Original design	Persistence was drawn as repositories only, and database setup was not shown.
Implemented change	Added a central database component, and supporting tables for data not drawn as classes, such as links, histories and saved results.
Reason	Uploads must be saved all-or-nothing, and several features need stored links, history or saved results.
Impact	Persistence only. No pattern change.

C07 — Repository-specific query methods

Original design	One generic repository interface for every type of stored data.
Implemented change	Kept the interface and added the extra queries individual features need.
Reason	Features need lookups beyond the generic operations, such as detecting duplicate files.
Impact	Persistence only. No pattern change.

C08 — AI-layer supporting types

Original design	The AI layer had a small set of request and response types, a prompt builder and a response parser.
Implemented change	Added types for conversation messages and AI errors, and extended the builder and parser, including one automatic repair of malformed replies.
Reason	The agent loop needs conversation history and tool results, and malformed replies should be handled in one place.
Impact	AI layer only. Builder and Strategy patterns unchanged.

C09 — Gemini adapter details

Original design	The adapter converts between StudyBuddy's request types and LangChain4j.
Implemented change	The adapter also retries temporary failures and converts tool definitions.
Reason	Every AI feature needs the same retry behaviour, and the Study Coach needs tools.
Impact	AI layer only. Adapter pattern unchanged.

C10 — Removing materials from search, and relevance threshold

Original design	Retrieval could index passages and return the best matches.
Implemented change	Retrieval can also remove a material, and returns only passages that are relevant enough.

Reason	Removed materials must leave the search index, and F03 must be able to say a question is not covered.
Impact	F01 and F03. Strategy pattern unchanged.

C11 — Course and material management responsibilities

Original design	MaterialService created courses and uploaded and split materials.
Implemented change	MaterialService also handles the rest of course and material management.
Reason	F01 includes renaming, removing, replacing duplicates and keeping materials searchable after a restart, and no other service owned these.
Impact	F01. The facade gained matching methods (C13).

C12 — Keyword fallback for embedding search

Original design	Embedding and keyword retrieval were independent alternatives.
Implemented change	Embedding retrieval falls back to keyword search when embeddings are unavailable.
Reason	A material must stay searchable if the embedding service fails.
Impact	F01 and F03. Strategy pattern unchanged.

C13 — Application assembly and facade growth

Original design	The facade holds all services, and no component was shown for building them.
Implemented change	Added one component that builds the application, and added facade methods for the extra management actions.
Reason	Keeps construction code out of the GUI and CLI.
Impact	Facade pattern unchanged. The updated class diagram shows the new component.

C14 — CLI structure and runnable jar

Original design	A CLI with subcommands.
Implemented change	Commands are organized by feature area, with JSON output and consistent exit codes, and the application is packaged as one runnable jar.
Reason	Management actions must be available from the CLI, and Stage 3 agent validation needs machine-readable results.
Impact	Presentation layer only.

C15 — Question history and general-knowledge answers

Original design	F03 called for a question history and labelled general-knowledge answers, but no class showed them.
Implemented change	Added storage for the question history, and answers record whether they came from general knowledge.
Reason	The history must be saved, and a general-knowledge answer must be distinguishable from a "not covered" reply.

Impact	F03 and F10.

C16 — Failure handling and running without a key

Original design	Question answering was designed around a working model.
Implemented change	Retrieved passages are still shown when the model fails, and a stand-in client is used when no API key is set.
Reason	Passages remain useful when the model fails, and deterministic features must keep working without a key.
Impact	F03, with F02 and F07 reusing the same handling. The Strategy pattern gains one more LLM client implementation.

C17 — Concept review and management

Original design	Concept extraction was a single step.
Implemented change	Extracting and saving are separate steps, and concepts can be edited, merged and deleted.
Reason	F02 requires the student to review concepts before saving and to manage the list afterwards.
Impact	F02, and F04, F05 and F08, which use concept tags. No pattern change.

C18 — Chunks group whole pages and record a page range

Original design	Text was split into overlapping chunks of a fixed size, each with one page number.
Implemented change	Consecutive whole pages are grouped into a chunk that records its page range, and only very long pages are split.
Reason	Found in live tests: citations gave the wrong page, and splitting every page separately produced too many chunks for the embedding rate limit.
Impact	F01, and citations in F02, F03 and F07.

C19 — No embedding calls at startup

Original design	Missing embeddings were requested again when a course was opened.
Implemented change	Startup never calls the API. Missing embeddings are retried when the GUI opens a course or through an explicit CLI command.
Reason	Every CLI command is a new process, so retrying at startup kept hitting the rate limit.
Impact	F01. Strategy pattern unchanged.

C20 — GUI structure

Original design	A main window with one panel per feature area.
Implemented change	Built as designed. The main window holds the course selector, and slow work runs in the background.
Reason	The GUI must stay responsive during AI calls and still report results and errors.

Impact	Presentation only. The main window subscribes to study events (Observer).

C21 — Shared GUI helpers

Original design	Panels would use standard Swing components.
Implemented change	Added shared helpers for consistent fonts, colours and table appearance.
Reason	The default font and table rendering looked inconsistent on Windows.
Impact	Presentation only. No effect on features or design patterns.

C22 — Quiz storage, retakes and extra quiz operations

Original design	Quizzes could be generated, started and submitted, and no storage was shown for attempts.
Implemented change	Attempts are stored separately, and students can retake quizzes, re-grade pending answers and dispute grades. A quiz is shortened when the material is too short.
Reason	F06 requires re-grading and disputes, and results must survive a restart.
Impact	F05 and F06 (UC06). Strategy and Factory patterns used as designed.

C23 — Weak topics renamed to weak concepts

Original design	F08 and its classes referred to "weak topics".
Implemented change	Renamed to "weak concepts" throughout. Scores are calculated when needed and not stored.
Reason	Each result describes one concept, and the application has no separate kind of topic.
Impact	F08 and F09 (UC07, UC09); MasteryService and GetWeakConceptsTool. No effect on design patterns.

C24 — Flashcard operations and shared scope

Original design	Flashcards were generated from concepts and reviewed in a separate window, and F08 offered practice shortcuts on the Progress tab.
Implemented change	Decks can be generated from materials or concepts and edited in a preview before saving. Review happens inside the Flashcards tab. Quizzes and flashcards share one way of choosing content, and decks hold 5 to 20 cards. The Progress tab has no practice shortcuts, because the Flashcards and Quizzes tabs do the same job.
Reason	A preview needs generating and saving to be separate, shared logic should exist once, and the deck limit keeps model output manageable.
Impact	F04, F05 and F08 (UC05); FlashcardService, StudyScope and FlashcardRepository. No effect on design patterns.

C25 — Study Coach details and model

Original design	The Coach used a larger Gemini model and four tools.

Implemented change	The Coach uses the same low-cost model as the other features. A fifth tool saves lasting preferences, steps are shown as they happen, and every tool call counts towards the step limit. The adapter returns the extra data Gemini requires after a tool call, and a failed run keeps the work already done.
Reason	The larger model was too slow and allowed too few free requests, while the low-cost model handled tool calling correctly in live tests. Counting every call keeps model use bounded.
Impact	F09 (UC04); AgentController, Planner and SavePreferenceTool. Command pattern unchanged.

C26 — Explanations without follow-up questions

Original design	F07 let the student ask a follow-up question.
Implemented change	The follow-up question is left out. Explanations are saved with the attempt and cleared if the grade is disputed.
Reason	The explanation itself meets the student's need, saving it avoids repeat model calls, and a changed grade makes the old explanation out of date.
Impact	F07 (UC08); LearningContentService and QuizPanel. No effect on design patterns.

C27 — Progress without a chart or dashboard

Original design	A dashboard with summary cards, and a Progress tab with a score chart, mastery and history.
Implemented change	There is no dashboard or chart. The Progress tab shows the average score and streak above concept mastery and the study history, which also lists items created.
Reason	Cards due are already shown in the Flashcards tab, dated scores in the history say as much as a chart over a few quizzes, and items the Coach creates must appear in the history.
Impact	F10 (UC07); ProgressService. The Observer pattern gains a second listener.

C28 — Coach history storage and consistent controls

Original design	A general activity log, materials managed through a right-click menu, and tables that allowed several selected rows.
Implemented change	The log now records only coach conversations, the Materials and Concepts tabs work the same way, and tables allow one selected row.
Reason	The log only ever held coach conversations, the two tabs should behave alike, and every table action works on one row.
Impact	F01, F02 and F10. No effect on design patterns.

C29 — Flashcards without source pages, deck deletion, no coach links, deselecting

Original design	Cards showed a source page and schedule box, decks could not be deleted, items the Coach created appeared as links, and a selection could not be cleared.
Implemented change	Cards show their concept and next due date instead. Decks can be deleted while their ratings are kept, the Coach names what it created, and clicking a selected row clears it.
Reason	A page number means little without its material, mastery and history must survive deleting a deck, and the links saved little.
Impact	F04, F09 and F10 (UC04, UC05); FlashcardService and CoachPanel. No effect on design patterns.

C30 — Concept review can be cancelled

Original design	A concept review could only end by saving.
Implemented change	Added a Cancel button that restores the saved concepts, and extraction is disabled during a review.
Reason	There was no way out of a review, and repeated clicks started repeat model calls.
Impact	F02 (UC02). No effect on design patterns.

C31 — CLI command renamed to studybuddy, launchers

Original design	The CLI command was called studyagent.
Implemented change	The command is now studybuddy, and launcher scripts were added.
Reason	One name for the application, the command and the launcher.
Impact	CLI usage text and README only.

5. AI Tools and Collaboration Log

AI Tools

Tool or platform	LLM/model	Features, classes or tasks	Total token cost	Money cost
Claude Code (desktop app)	Claude Opus 5.5 (claude-opus-5-5)	All of F01 to F10 and every package in src/; JUnit tests; README, launchers and sample PDFs; UML regeneration from source; this report's first draft	269,802,162 (from the Claude Code session log, as of Nov 8)	$28 (Pro tier)
Google Gemini API	gemini-3.1-flash-lite, gemini-embedding-001	Live end-to-end checks and hands-on use of each AI feature against the real API	738,219 (from the AI Studio usage page, Oct 12 to Nov 8)	$0 (free tier)

How AI and Human Work Were Divided

The developer set the scope, priority order and every feature decision, including those recorded in Section 4. Claude Code wrote the code and tests against the Stage 1 design and ran the build. The developer reviewed the work, tried the features in the running application and made every commit personally.

Claude Code worked from written design documents and a set of project rules: keep the Stage 1 class names, log every deviation from the design, keep model libraries inside the AI layer, never commit secrets, and never call the real API from tests.

Representative Interactions

Date	Task	Model
2026-10-26	Project setup and foundations: Maven build and wrapper, AppConfig, domain model, SQLite Database and schema, Repository<T>, course and material repositories, unit tests	Claude Opus 5.5
2026-10-29	AI integration layer: LLMClient, FakeLLMClient, PromptBuilder, ResponseParser, GeminiClient adapter, DocumentParser, KeywordRetriever, tests	Claude Opus 5.5
2026-10-29	F01 ingestion and application shell: EventBus, MaterialService, EmbeddingRetriever, StudyAgentFacade, ApplicationFactory, course and material commands, runnable jar, tests	Claude Opus 5.5
2026-10-30	F03 question answering and F02 concept extraction: QaHistoryRepository, answerQuestion, NoKeyLLMClient, ConceptRepository, extraction and duplicate merging, ask and concepts commands, tests, live tests	Claude Opus 5.5
2026-10-30	Chunking reworked to group whole pages with page ranges, embedding calls removed from startup, material reindex command, tests, live check	Claude Opus 5.5
2026-11-06	Swing GUI: MainWindow, FeaturePanel, Materials, Ask and Concepts panels, gui command, Theme and StudyTable	Claude Opus 5.5
2026-11-06	F05 quiz generation and F06 grading: QuestionFactory, graders, repositories, QuizService, quiz commands, QuizPanel, tests, live check	Claude Opus 5.5; live check used a few gemini-3.1-flash-lite calls
2026-11-06	F08 weak concepts: rename from weak topics, MasteryService, weak-concepts command, ProgressPanel, tests, live check	Claude Opus 5.5; about 10 gemini-3.1-flash-lite calls
2026-11-06	F04 flashcards: StudyScope, FlashcardRepository, FlashcardService, flashcards commands, FlashcardPanel, ratings as mastery evidence, tests, live check	Claude Opus 5.5; about 6 gemini-3.1-flash-lite calls
2026-11-06	F09 Study Coach: agent package, tools, preference storage, coach command, CoachPanel, thought signature fix, tests, live checks	Claude Opus 5.5; about 50 gemini-3.1-flash-lite calls
2026-11-06	F07 explanations: ExplanationRepository, explainAnswer, explain command, Explain button, tests, live check	Claude Opus 5.5; about 6 gemini-3.1-flash-lite calls
2026-11-06	F10 progress and history: ProgressService, report export, coach history line, progress and history commands, Progress tab, tests, live check	Claude Opus 5.5; about 10 gemini-3.1-flash-lite calls

Date	Task	Model
2026-11-06 to 11-07	GUI and workflow refinements: consistent Materials and Concepts controls, deck deletion that keeps ratings, concept review Cancel, click to deselect, tests, live checks	Claude Opus 5.5; about 9 gemini-3.1-flash-lite calls and 1 embedding
2026-11-07	README, launchers and sample PDFs; README example session run end to end on the samples	Claude Opus 5.5; about 32 gemini-3.1-flash-lite calls and 5 embedding batches
2026-11-08	UML updated to match the code: class diagram regenerated from source, all six sequence diagrams rebuilt	Claude Opus 5.5
2026-11-08	First draft of the Stage 2 report from the changes log, AI log, git history and test classes	Claude Opus 5.5

6. Development and Verification Summary

Weekly Commit Summary

Dates are Toronto local time. Commit counts include every commit in the period in the repository.

Period	Dates	Commits	Summary of work
1	Oct 20–26	7	Maven build and Maven Wrapper, AppConfig settings loader, domain model (enums, value types, entities, quiz domain), SQLite Database and schema, and the Repository interface with course and material repositories.
2	Oct 27–Nov 2	20	AI integration layer (LLMClient, PromptBuilder, ResponseParser, GeminiClient adapter), document parsing, keyword and embedding retrieval, EventBus, MaterialService (F01), StudyAgentFacade and the CLI. F03 question answering and F02 concept extraction through the CLI.
3	Nov 3–9	34	Swing GUI (MainWindow, FeaturePanel and all tabs). F05 and F06 quizzes and grading, F08 weak concepts, F04 flashcards, F09 Study Coach, F07 explanations and F10 progress and history. GUI fixes, deck deletion, README, launchers, sample materials and updated UML.

Checks Performed

Automated tests. ./mvnw verify compiles the project, runs the JUnit 5 suite and builds the runnable jar. The final run passes all 371 tests in 47 test classes. Tests were written with each step of the build, as listed below. They use FakeLLMClient with scripted responses, in-memory or temporary SQLite databases, and never call the real API.

Development step	Test classes	Tests	What they check
Configuration and domain model	AppConfigTest, CourseEntitiesTest, ValueTypesTest, QuizDomainTest	23	Settings precedence, the key never appearing in output, Leitner scheduling, question validation, attempt state

Development step	Test classes	Tests	What they check
Persistence	DatabaseTest, CourseAndMaterialRepositoryTest	10	Schema creation, transaction rollback, foreign-key cascades, unique course names, duplicate lookup
AI integration	PromptBuilderAndFakeClientTest, ResponseParserTest, GeminiClientTest	21	Prompt assembly, JSON repair, citation checks, retries, tool-call conversion, thought signatures
Ingestion and retrieval (F01)	DocumentParserTest, KeywordRetrieverTest, EmbeddingRetrieverTest, MaterialServiceTest, EventBusTest	37	Text cleaning, ranking, keyword fallback, chunking, duplicates, event delivery
Facade and CLI	StudyAgentFacadeTest, StudyAgentCliTest	6	Data surviving a restart, keyword search without a key, JSON output and exit codes
F03 and F02	LearningContentServiceTest, ConceptExtractionTest, QaHistoryRepositoryTest, ConceptRepositoryTest, AskCommandTest, ConceptsCommandTest	24	Grounded and not-covered answers, duplicate merging, concept merge and delete
GUI shell	PanelsSmokeTest, MaterialsPanelTest, ConceptsPanelTest, DeselectTest	17	Events refreshing panels, rename and remove, the concept review flow, clearing a selection
F05 and F06	QuestionFactoryTest, GradersTest, QuizRepositoryTest, QuizServiceTest, QuizCommandTest, QuizPanelTest	68	Question creation and validity, regeneration, rule and rubric grading, pending answers, disputes
F08	MasteryServiceTest, WeakConceptsCommandTest	28	Score weighting, trends, insufficient data, diagnosis failure
F04	FlashcardRepositoryTest, FlashcardServiceTest, FlashcardsCommandTest, FlashcardPanelTest	42	Card validation, deck saving and deletion, due cards, ratings kept after a deck is deleted
F09	ToolManagerTest, AgentControllerTest, CoachCommandTest, CoachPanelTest	49	Argument validation, tool execution, step limit, failures partway through, memory, preferences
F07	ExplanationTest, ExplainCommandTest	15	Caching, missing source, misconception rules, pending answers rejected
F10	ProgressServiceTest, ProgressCommandTest, ProgressPanelTest	31	History rows and filters, streak, report export, out-of-order background results

Live checks. After each AI feature was built, it was run against the real Gemini API on real course material. Most features took about 6 to 10 model calls, and the Study Coach took about 50. These checks found problems the scripted tests could not, as listed in the defects table.

Application checks. GUI behaviour was exercised in the running window, including file dialogs and table rendering at fractional display scaling. The example session in the README was run from start

to finish on the three PDFs in samples/. .gitignore covers .env, .env.*, config.local.json and the local database.

Important Defects Found and Corrected

Defect	How it was found	Correction	Change
Citations said "p. 1" for text on later pages, because the sliding-window chunker crossed pages but recorded only the first	Live test	Chunks group whole pages and record an exact page range	C18
Chunking each page separately turned slide decks into about six times as many chunks and exceeded the embedding rate limit	Live test	Consecutive pages are grouped up to 400 words	C18
Every CLI command requested missing embeddings at startup, so one failure kept hitting the rate limit	Live test	Startup never calls the API; retrying is an explicit reindex	C19
Gemini rejected the follow-up to a tool call with HTTP 400	Live test of the Study Coach	Each tool call's thought signature is sent back unchanged	C25
The planned Coach model allowed 20 requests a day and took 1–3 minutes per call	Live test	The Coach uses gemini-3.1-flash-lite, which handled tool calling correctly	C25
With concepts chosen, questions and cards could drift to other concepts	Live test	Generated items must be tagged with one of the chosen concepts	C24
A changed grade could leave a misconception on an answer now counted as correct	Review	Disputing a grade clears that question's cached explanation	C26
Deleting a deck would have removed its ratings, and so changed mastery and the streak	Review	Ratings outlive their deck (ON DELETE SET NULL)	C29
Repeated clicks during concept review started repeat model calls, and there was no way out except removing every row	GUI testing	Extract is disabled during a review, and Cancel was added	C30
Older background results could overwrite newer ones in the Progress tab	An intermittent test failure	Only the newest request of each kind may update the panel	—
Table grid lines measured 1 or 2 pixels depending on the row at fractional display scaling	GUI testing	StudyTable paints its own one-pixel lines	C21

Remaining Limitations

In addition to the limitations in Section 1:

- There are no database migrations, so a database from an older schema must be deleted and recreated.
- The Gemini free tier limits throughput, so uploading large materials can leave some embeddings to a later reindex, and the service was occasionally overloaded (HTTP 503) during testing.

7. Demonstration Index

Each implemented feature is linked below to its demonstration video or labelled video segment.

Feature ID and name	What the demonstration shows	Video link
F01 — Course and Material Management	Create a course, upload a sample PDF, duplicate upload, rename and remove	https://drive.google.com/file/d/1RgyOD2rMaxIQczI9JrvJszfDBlFBEd16/view?usp=drive_link
F02 — Concept Extraction	Extract concepts, edit the candidates, save, merge	https://drive.google.com/file/d/19VvBUBV-e47xMJVX4pg-9yPnl3pS_EFg/view?usp=drive_link
F03 — Question Answering over Course Materials	Ask a covered question with citations, then one the materials do not cover	https://drive.google.com/file/d/17FxRR8xniGjY8xbYvJgPmJty9LO041en/view?usp=drive_link
F04 — Flashcard Generation and Review	Generate and edit a deck, review and rate cards, delete a deck	https://drive.google.com/file/d/1s528BwL6EAiA_69QviwCCLh6b1lhpru0/view?usp=drive_link
F05 — Quiz Generation	Choose scope, types and difficulty, and generate a quiz	https://drive.google.com/file/d/1z44zFqFwg1hX-y89M0f-X3_p_dNJaY3D/view?usp=drive_link
F06 — Quiz Grading	Submit answers, see rule and AI grades with feedback, dispute a grade	https://drive.google.com/file/d/1-88aNDoQvLIArPuq-KdLW7YLq938aB9_/view?usp=drive_link
F07 — Answer Explanation	Explain a wrong answer, with its citation	https://drive.google.com/file/d/1mXrjssd6hehjI0-fzBRq7pfI6yA_qmPJ/view?usp=drive_link
F08 — Weak-Concept Identification	View ranked mastery and weak concepts, run the diagnosis	https://drive.google.com/file/d/1TOZIn4fsf2VBEYOKSLTthuHszY3MihO2/view?usp=drive_link
F09 — Study Coach	State a goal, watch the steps and tool calls, see what was created	https://drive.google.com/file/d/17RCoJIDfhiUHBQLTR-tr1ivJvGobppDo/view?usp=drive_link
F10 — Progress Tracking and Study History	View the summary and history, filter it, export a report	https://drive.google.com/file/d/1GhCmjxU_tNwq9BItWallPFxMtsu9vWar/view?usp=drive_link